```
# Imports pandas as 'pd'. Pandas is the standard library used for data manipulation and analysis, specifically for handling tabu
import pandas as pd

# Imports numpy as 'np'. Numpy provides support for large, multi-dimensional arrays and mathematical functions to operate on the
import numpy as np

# Imports the 're' module for Regular Expressions, which is essential for pattern matching and cleaning messy string text. [cite
import re

# Imports the pyplot interface of matplotlib. This is traditionally used for plotting graphs and visual charts. [cite: 6]
import matplotlib.pyplot as plt

# Imports LogisticRegression. This is a fundamental linear model used for binary classification, replacing the tree-based XGBoost
from sklearn.linear_model import LogisticRegression

# Imports the XGBoost Classifier algorithm. This is a powerful, tree-based machine learning model used here for our primary class
from xgboost import XGBClassifier

# Imports functions from scikit-learn to evaluate model performance: accuracy (% correct), detailed metrics, and error breakdown
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Imports the tool to convert raw text into numerical features using Term Frequency-Inverse Document Frequency (TF-IDF). [cite: 9
from sklearn.feature_extraction.text import TfidfVectorizer

# Imports tools to scale numeric data to a standard range (StandardScaler) and convert categorical text labels into integer numbe
from sklearn.preprocessing import StandardScaler, LabelEncoder

# Imports the function to randomly split datasets into training and testing subsets to evaluate model generalization. [cite: 11]
from sklearn.model_selection import train_test_split

# Imports the pickle module, used for serializing (saving) and deserializing (loading) Python objects like trained models to the
import pickle

# Imports TensorFlow, the core deep learning library used here for building the Autoencoder neural network. [cite: 13]
import tensorflow as tf

# Imports the 'Model' class to define custom neural network architectures, and 'load_model' to retrieve saved models from disk.
from tensorflow.keras.models import Model, load_model

# Imports specific neural network layer types: 'Input' for the entry point, and 'Dense' for standard fully connected layers. [cit
from tensorflow.keras.layers import Input, Dense

# Imports the drive module specific to Google Colab to allow the environment to access files stored in your Google Drive. [cite:
from google.colab import drive

# ========================= DATA LOADING =========================
# Mounts Google Drive to the Colab environment at '/content/drive'.
# Parameter 'force_remount=True' forces a fresh connection, preventing errors from stale or timed-out connections. [cite: 18]
drive.mount("/content/drive", force_remount=True)

# Reads the depression dataset from a CSV file into a pandas DataFrame named 'text_df'.
# Parameter filepath points to the file location in Drive.
# Parameter 'encoding="utf-8"' ensures text with special characters or emojis is decoded properly. [cite: 19]
text_df = pd.read_csv("/content/drive/MyDrive/dataset/MentalHealth/depression_dataset_reddit_cleaned.csv", encoding="utf-8")


# Reads the student mental health dataset from a CSV into 'tab_df' using standard utf-8 encoding. [cite: 20]
tab_df  = pd.read_csv("/content/drive/MyDrive/dataset/MentalHealth/Student Mental health.csv", encoding="utf-8")

# Prints the first 5 rows of 'text_df' to visually verify the text data loaded correctly. [cite: 21]
print(text_df.head())

# Prints the first 5 rows of 'tab_df' to visually verify the tabular survey data loaded correctly. [cite: 22]
print(tab_df.head())
```

```
Mounted at /content/drive
                                          clean_text  is_depression
0  we understand that most people who reply immed...              1
1  welcome to r depression s check in post a plac...              1
2  anyone else instead of sleeping more when depr...              1
3  i ve kind of stuffed around a lot in my life d...              1
4  sleep is my greatest and most comforting escap...              1
        Timestamp Choose your gender   Age What is your course?  \
0  8/7/2020 12:02             Female  18.0          Engineering
1  8/7/2020 12:04               Male  21.0    Islamic education
2  8/7/2020 12:05               Male  19.0                  BIT
```

```
3  8/7/2020 12:06             Female  22.0                 Laws
4  8/7/2020 12:13               Male  23.0         Mathemathics

  Your current year of Study What is your CGPA? Marital status  \
0                     year 1        3.00 - 3.49            No
1                     year 2        3.00 - 3.49            No
2                     Year 1        3.00 - 3.49            No
3                     year 3        3.00 - 3.49           Yes
4                     year 4        3.00 - 3.49            No

  Do you have Depression? Do you have Anxiety? Do you have Panic attack?  \
0                     Yes                  No                       Yes
1                      No                 Yes                        No
2                     Yes                 Yes                       Yes
3                     Yes                  No                        No
4                      No                  No                        No

  Did you seek any specialist for a treatment?
0                                           No
1                                           No
2                                           No
3                                           No
4                                           No
```

```
# ========================= TEXT PREPROCESSING =========================
# Defines a custom function named 'clean_text' taking a single parameter 'text' (a string). [cite: 96]
def clean_text(text):
    # Converts the input to a string (handling potential nulls) and changes all characters to lowercase so capitalization doesr
    text = str(text).lower()

    # Uses regex (re.sub) to replace web URLs with an empty space (' ').
    # Literal r'http\S+|www\S+': matches 'http' or 'www' followed by non-whitespace characters (\S+). [cite: 98]
    text = re.sub(r'http\S+|www\S+', ' ', text)

    # Uses regex to find and remove user mentions starting with '@' followed by word characters (\w+). [cite: 99]
    text = re.sub(r'@\w+', ' ', text)

    # Uses regex to find and remove hashtags starting with '#' followed by word characters (\w+). [cite: 100]
    text = re.sub(r'#\w+', ' ', text)

    # Uses regex to remove any character that is NOT (^) a letter (a-z, A-Z) or a space (\s), dropping numbers and punctuation.
    text = re.sub(r'[^a-zA-Z\s]', ' ', text)

    # Replaces multiple consecutive spaces (\s+) with a single space (' ') and strips leading/trailing spaces from the string e
    text = re.sub(r'\s+', ' ', text).strip()

    # Returns the fully cleaned text string back to the caller. [cite: 104]
    return text

# Recreates the dataframe using only the needed columns. '.copy()' prevents pandas memory reference warnings. [cite: 105]
text_df = text_df[['clean_text', 'is_depression']].copy()

# Applies the custom 'clean_text' function to every row in the 'clean_text' column, overwriting the raw data. [cite: 106]
text_df['clean_text'] = text_df['clean_text'].apply(clean_text)

# Initializes the TF-IDF Vectorizer to convert cleaned text strings into numerical matrices. [cite: 107]
tfidf = TfidfVectorizer(
    # Limits the vocabulary to the top 10,000 most frequent words to save memory and ignore rare noise. [cite: 107]
    max_features=10000,
    # Extracts both single words (unigrams, 1) and two-word combinations (bigrams, 2) to capture context. [cite: 107]
    ngram_range=(1, 2),
    # Automatically removes common English words ('the', 'is') that carry little predictive value. [cite: 107]
    stop_words='english',
    # Ignores words appearing in fewer than 2 documents (filters out extreme typos). [cite: 107]
    min_df=2,
    # Ignores words appearing in more than 95% of documents (filters out dataset-specific stop words). [cite: 107]
    max_df=0.95,
    # Applies logarithmic scaling to term frequencies to prevent highly repeated words from dominating the math. [cite: 107]
    sublinear_tf=True
)

# Learns the vocabulary (.fit) and transforms the text into a numeric sparse matrix representing TF-IDF scores. Assigns to 'X_t
X_text = tfidf.fit_transform(text_df['clean_text'])

# Extracts the target labels (1 for depression, 0 for not) and assigns them to 'y_text'. [cite: 112]
y_text = text_df['is_depression']
```

```
# ========================= TABULAR PREPROCESSING =========================
# Creates an explicit copy of the tabular dataframe so modifications don't corrupt the original reference. [cite: 117]
tab_df = tab_df.copy()

# Replaces cells containing only spaces (r'^\s*$') with Numpy's 'NaN' (Not a Number) to standardize missing values. regex=True
tab_df = tab_df.replace(r'^\s*$', np.nan, regex=True)

# Removes any row (.dropna) containing a NaN value, then resets the row index sequentially dropping the old index (drop=True).
tab_df = tab_df.dropna().reset_index(drop=True)

# Checks if the column literal 'Timestamp' exists in the dataframe. [cite: 121]
if 'Timestamp' in tab_df.columns:
    # Drops the 'Timestamp' column entirely (axis=1 means column-wise) because submission time doesn't predict mental health. [
    tab_df = tab_df.drop('Timestamp', axis=1)

# Defines a function to clean categorical pandas Series. [cite: 123]
def clean_categorical(col):
    # Converts all items to strings, makes them lowercase, and removes extra spaces to ensure "Yes " and "yes" are equal. [cite
    return col.astype(str).str.lower().str.strip()

# Iterates through every column name in the dataframe. [cite: 125]
for col in tab_df.columns:
    # Checks if the column data type is 'object' (which represents text/strings in pandas). [cite: 126]
    if tab_df[col].dtype == 'object':
        # Applies the categorical cleaning function to standardize the text format in that column. [cite: 127, 129]
        tab_df[col] = clean_categorical(tab_df[col])

# Initializes an empty dictionary to store LabelEncoder objects for later reuse during inference. [cite: 128]
label_encoders = {}

# Iterates again through all columns. [cite: 130]
for col in tab_df.columns:
    # If the column contains text data... [cite: 131]
    if tab_df[col].dtype == 'object':
        # Instantiates a LabelEncoder to map text categories to integer values (e.g., 'no' -> 0). [cite: 132]
        le = LabelEncoder()
        # Learns the unique mapping (.fit) and replaces the text in the dataframe with integers (.transform). [cite: 133]
        tab_df[col] = le.fit_transform(tab_df[col])
        # Saves the fitted encoder into the dictionary using the column name as the key. [cite: 134]
        label_encoders[col] = le

# Defines the literal name of the target column we are trying to predict. [cite: 135]
target_col = 'Do you have Depression?'

# Creates the feature set 'X_tab' by dropping the target column. [cite: 138]
X_tab = tab_df.drop(target_col, axis=1)

# Extracts only the target column to serve as the label set 'y_tab'. [cite: 139]
y_tab = tab_df[target_col]

# Saves the names of the feature columns into a Python list for tracking purposes. [cite: 142]
feature_columns = X_tab.columns.tolist()

# Instantiates a StandardScaler to normalize features so they have a mean of 0 and standard deviation of 1. [cite: 143]
scaler = StandardScaler()

# Calculates mean/std (.fit) and applies the mathematical scaling (.transform) to the tabular data. [cite: 144]
X_tab_scaled = scaler.fit_transform(X_tab)
```

```
# ========================= TRAIN/TEST SPLIT =========================
# Splits the text dataset into training (80%) and testing (20%) subsets. [cite: 149]
# Parameter test_size=0.2: Dictates that 20% of the data goes to testing. [cite: 149]
# Parameter random_state=42: A fixed integer seed ensuring the exact same random split occurs every time the code runs. [cite:
# Parameter stratify=y_text: Ensures the ratio of positive to negative classes remains equal in both train and test sets. [cite
X_train, X_test, y_train, y_test = train_test_split(X_text, y_text, test_size=0.2, random_state=42, stratify=y_text)

# Performs the exact same 80/20 stratified split operation, but for the scaled tabular data. [cite: 150]
X_train_t, X_test_t, y_train_t, y_test_t = train_test_split(X_tab_scaled, y_tab, test_size=0.2, random_state=42, stratify=y_tab
```

```
# ========================= XGBoost TEXT MODEL =========================
# Initializes the XGBoost classifier for the NLP (text) data. [cite: 152]
text_model = XGBClassifier(
    # Parameter scale_pos_weight: Corrects class imbalance by assigning higher weight to the minority class. Calculated as total
    scale_pos_weight=len(y_train)/sum(y_train),
```

```
    # Parameter random_state=42: Ensures tree-building randomness is reproducible. [cite: 155]
    random_state=42,
    # Parameter n_estimators=300: Instructs the algorithm to build exactly 300 sequential decision trees. [cite: 156]
    n_estimators=300,
    # Parameter max_depth=6: Limits how deep each tree can grow to 6 splits, preventing overfitting. [cite: 157]
    max_depth=6,
    # Parameter learning_rate=0.1: Applies a 10% step size shrinkage during weight updates to improve robustness. [cite: 158]
    learning_rate=0.1,
    # Parameter eval_metric='logloss': Sets Logarithmic Loss as the function to evaluate model performance during training. [cite
    eval_metric='logloss'
)

# Trains the model using the text training features and their corresponding labels. [cite: 160]
text_model.fit(X_train, y_train)

# Uses the trained model to generate predictions (0 or 1) on the unseen text test set. [cite: 161]
pred = text_model.predict(X_test)

# Calculates and prints the ratio of correct predictions against total predictions using the accuracy_score function. [cite: 162
print("Text XGBoost Accuracy:", accuracy_score(y_test, pred))


# Initializes a separate XGBoost classifier specifically for the tabular data. [cite: 166]
tabular_model = XGBClassifier(
    # Parameter scale_pos_weight: Class imbalance correction applied to the tabular labels. [cite: 168]
    scale_pos_weight=len(y_train_t)/sum(y_train_t),
    # Parameter random_state=42: Reproducibility seed. [cite: 169]
    random_state=42,
    # Parameter n_estimators=200: Builds 200 trees (fewer than text model due to lower feature complexity). [cite: 170]
    n_estimators=200,
    # Parameter max_depth=5: Restricts trees to a depth of 5. [cite: 171]
    max_depth=5,
    # Parameter learning_rate=0.1: Standard 10% shrinkage factor. [cite: 172]
    learning_rate=0.1,
    # Parameter eval_metric='logloss': Evaluates using Logarithmic Loss. [cite: 173]
    eval_metric='logloss'
)

# Trains the model exclusively on the tabular training data. [cite: 174]
tabular_model.fit(X_train_t, y_train_t)

# Generates predictions on the unseen tabular test set. [cite: 175]
pred2 = tabular_model.predict(X_test_t)

# Calculates and prints the accuracy for the tabular model. [cite: 176]
print("Tabular XGBoost Accuracy:", accuracy_score(y_test_t, pred2))
```

```
Text XGBoost Accuracy: 0.9553975436328378
Tabular XGBoost Accuracy: 0.75
```

Start coding or generate with AI.

```
# ========================= XGBoost TABULAR MODEL =========================
tabular_model = XGBClassifier(
    scale_pos_weight=len(y_train_t)/sum(y_train_t),
    random_state=42,
    n_estimators=200,
    max_depth=5,
    learning_rate=0.1,
    eval_metric='logloss'
)
tabular_model.fit(X_train_t, y_train_t)
pred2 = tabular_model.predict(X_test_t)
print("Tabular XGBoost Accuracy:", accuracy_score(y_test_t, pred2))
```

```
Tabular XGBoost Accuracy: 0.75
```

```
# ============================ AUTOENCODER (Anomaly Detection) ====================================
# ==== This builds a neural network that compresses and reconstructs data to find "unusual" patterns.===

# Dynamically extracts the number of columns from the tabular data to define the size of the neural network's input layer. [cit
input_dim = X_train_t.shape[1]
```

```
# Defines the entry point of the neural network. Parameter shape=(input_dim,) means it accepts 1D arrays of that length. [cite:
input_layer = Input(shape=(input_dim,))

# ENCODER: Creates a dense (fully connected) layer compressing the input down to 16 neurons.
# Parameter activation='relu': Applies Rectified Linear Unit math to introduce non-linearity. Takes 'input_layer' as its source
encoded = Dense(16, activation='relu')(input_layer)

# Bottleneck layer: Further compresses the data down to a tiny 8-neuron representation. [cite: 187]
encoded = Dense(8, activation='relu')(encoded)

# DECODER: Attempts to expand the 8-neuron bottleneck back up to 16 neurons. [cite: 188]
decoded = Dense(16, activation='relu')(encoded)

# Final output layer: Expands back to the exact original 'input_dim' size.
# Parameter activation='linear': Outputs direct continuous numeric values rather than scaling them between 0 and 1. [cite: 189]
decoded = Dense(input_dim, activation='linear')(decoded)

# Constructs the complete Keras Model object by linking the defined inputs to the defined outputs. [cite: 191]
autoencoder = Model(inputs=input_layer, outputs=decoded)

# Configures the model for training.
# Parameter optimizer='adam': Uses an adaptive gradient descent algorithm to update network weights.
# Parameter loss='mse': Uses Mean Squared Error to measure how badly the model failed to reconstruct the original input. [cite:
autoencoder.compile(optimizer='adam', loss='mse')

# Executes the training process. Notice X_train_t is passed as BOTH input and target (it learns to copy itself).
# Parameter epochs=50: The model will view the entire dataset 50 times.
# Parameter batch_size=16: The model updates its internal weights after reviewing every 16 rows.
# Parameter validation_data: Uses the test set to monitor loss without training on it.
# Parameter verbose=0: Hides the Keras training progress bar from the console. [cite: 192, 198]
autoencoder.fit(X_train_t, X_train_t, epochs=50, batch_size=16, validation_data=(X_test_t, X_test_t), verbose=0)

# Commands the trained model to generate the reconstructed versions of the training data. [cite: 193]
recon_train = autoencoder.predict(X_train_t, verbose=0)

# Calculates the Reconstruction Error (Mean Squared Error per row).
# np.power(..., 2) squares the differences. np.mean(..., axis=1) averages these squares across all columns for a single student
train_mse = np.mean(np.power(X_train_t - recon_train, 2), axis=1)

# Sets the anomaly threshold using np.percentile at the literal value 95.
# Meaning the top 5% highest error rates in the training data establish the cutoff for what is considered an "anomaly". [cite:
threshold = np.percentile(train_mse, 95)

# Prints the mathematically calculated anomaly threshold float value to the console. [cite: 195]
print("Anomaly Threshold:", threshold)
```

```
Anomaly Threshold: 1.2044050200366712
```

```
# ========================= SAVE ALL ARTIFACTS =========================
# The python 'with open(filename, "wb") as f:' syntax safely opens a file in "Write Binary" mode.
# pickle.dump(object, f) serializes the complex Python object into a raw byte stream and saves it to the disk.

# Serializes and saves the trained XGBoost text classifier to the literal filename "text_model.pkl". [cite: 203]
with open("text_model.pkl", "wb") as f: pickle.dump(text_model, f)

# Serializes and saves the trained XGBoost tabular classifier to "tabular_model.pkl". [cite: 204]
with open("tabular_model.pkl", "wb") as f: pickle.dump(tabular_model, f)

# Serializes and saves the TF-IDF vectorizer (crucial to transform future text exactly how the model was trained). [cite: 205]
with open("tfidf_vectorizer.pkl", "wb") as f: pickle.dump(tfidf, f)

# Serializes and saves the StandardScaler instance (needed to apply the exact same mathematical scale to future data). [cite: 2
with open("scaler.pkl", "wb") as f: pickle.dump(scaler, f)

# Serializes and saves the dictionary containing all fitted LabelEncoders for categorical columns. [cite: 207]
with open("label_encoders.pkl", "wb") as f: pickle.dump(label_encoders, f)

# Serializes and saves the Python list of feature column names. [cite: 208, 210]
with open("feature_columns.pkl", "wb") as f: pickle.dump(feature_columns, f)

# Serializes and saves the calculated numeric anomaly threshold boundary. [cite: 209, 210]
with open("threshold.pkl", "wb") as f: pickle.dump(threshold, f)

# Uses the built-in Keras '.save()' function to export the Autoencoder architecture, weights, and optimizer state as an HDF5 fi
autoencoder.save("autoencoder_model.h5")
```

```
# Prints a final literal confirmation string indicating the script finished saving everything successfully. [cite: 212]
print(" All models and artifacts saved successfully!")
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format
 All models and artifacts saved successfully!
```